Completely remove the Resend email integration from the existing AI Legal application and replace it with a Gmail SMTP-based email system.

Do not use Resend anywhere after this change.

Do not ask users for a Resend API key.

Do not display a Resend API key configuration screen.

Do not create a Resend connector.

Use the existing Flask backend and Gmail SMTP for all email functionality.

IMPORTANT: Do not break any existing AI Legal functionality.

---

1. REMOVE RESEND COMPLETELY

Search the entire project for all Resend-related implementation and remove/replace it.

Remove:

```
RESEND_API_KEY
RESEND_FROM_EMAIL
RESEND_FROM_NAME
```

Remove:

- Resend SDK
- Resend API calls
- Resend connector
- Resend configuration UI
- Resend API key input
- Resend setup instructions
- Resend-specific error messages
- Resend frontend code
- Resend backend code
- Resend environment variables
- Any code that checks for a Resend API key

The application must no longer depend on Resend.

Do not leave an unused Resend integration that can interfere with email sending.

---

2. USE GMAIL SMTP

Implement email delivery using Gmail SMTP.

Use:

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USERNAME=website-email@gmail.com
SMTP_PASSWORD=GOOGLE_APP_PASSWORD
SMTP_FROM_EMAIL=website-email@gmail.com
SMTP_FROM_NAME=AI Legal
```

These values must be loaded from backend environment variables.

Do NOT hard-code credentials in the source code.

Do NOT store the Gmail password/App Password in the database.

Do NOT expose credentials to the frontend.

---

3. GMAIL APP PASSWORD

The application must use a Google App Password.

Do NOT use the normal Gmail password.

The administrator will configure:

```
SMTP_PASSWORD
```

with the Google App Password.

The App Password must never appear in:

- Frontend
- Browser storage
- API response
- Logs
- Database
- Error messages

---

4. ACTUAL EMAIL SENDER

This is extremely important.

Every email must be sent from the website's configured Gmail account.

For example:

```
From:
AI Legal <website-email@gmail.com>
```

The user must NEVER be able to change the actual From address.

The actual sender must always come from:

```
SMTP_FROM_EMAIL
```

The logged-in user's entered email must NOT replace the website's email address.

---

5. USER SENDER INFORMATION

When a user sends an email, provide these fields:

```
Recipient Email
Sender Name
Sender Email
Subject
Message
Attachment
```

The user's Sender Name and Sender Email are informational only.

Example:

```
Sender Name:
Pati Sravya

Sender Email:
sravya@gmail.com
```

The actual email must still be:

```
From:
AI Legal <website-email@gmail.com>
```

---

6. EMAIL BODY

Display the user's sender information inside the email body.

Example:

```
AI LEGAL

Sender Details
----------------------------

Name: Pati Sravya
Email: sravya@gmail.com

Message
----------------------------

Please find the requested legal document attached.

----------------------------

This email was sent through the AI Legal platform.
```

Create both:

- HTML email
- Plain-text fallback

The email should look professional and clean.

---

7. EMAIL FORM

Create/update the Send Email page.

Display:

```
SEND EMAIL

To Email
[_____________________________]
```

```
Sender Name
[_____________________________]

Sender Email
[_____________________________]

Subject
[_____________________________]

Message
[_____________________________]

Attachment
[ Select Generated Document ]

[ SEND EMAIL ]
```

Add this information below the form:

> Emails are sent from the official AI Legal website email. The sender name and email you enter will be displayed inside the email.

---

8. WHO CAN SEND EMAILS

Both authenticated roles must be able to send emails:

```
Normal User
Admin
```

Do not make email sending admin-only.

Unauthenticated users must not be able to send emails.

---

9. BACKEND API

Use:

```
POST /api/send-email
```

Accept:

```
{
  "to": "recipient@example.com",
  "sender_name": "Pati Sravya",
  "sender_email": "sravya@gmail.com",
  "subject": "Legal Document",
  "message": "Please find the requested document attached.",
  "document_id": 123
}
```

The backend must:

1. Check authentication.
2. Validate all input.
3. Check Gmail SMTP configuration.
4. Verify the requested document belongs to the logged-in user.
5. Create the email.
6. Use the website Gmail account as the From address.
7. Put the user's sender information inside the email body.
8. Attach the generated PDF if selected.
9. Send using Gmail SMTP.
10. Return a success response.
11. Record the email in email history.

---

10. SMTP IMPLEMENTATION

Use Python's secure email libraries.

Prefer:

```
import smtplib
from email.message import EmailMessage
import ssl
```

Use:

```
smtp.gmail.com
port 587
STARTTLS
```

Conceptually:

```
server = smtplib.SMTP(SMTP_HOST, SMTP_PORT)
server.starttls()
server.login(SMTP_USERNAME, SMTP_PASSWORD)
```

```
server.send_message(message)
server.quit()
```

Use proper exception handling and always close the SMTP connection.

Never disable TLS.

---

11. EMAIL HEADERS

The email headers must be:

```
From:
AI Legal <SMTP_FROM_EMAIL>

To:
user-entered recipient

Subject:
user-entered subject
```

Do NOT set:

```
From = sender_email
```

The user-entered sender email must only appear in the email body.

Do not automatically use the user-entered sender email as Reply-To.

---

12. LEGAL DOCUMENT PDF ATTACHMENT

Integrate the new Gmail system with the existing legal document generator.

After generating a legal document, provide:

```
Download PDF
Send by Email
```

When the user clicks:

```
Send by Email
```

open the email form and automatically select the generated document.

The generated PDF must be attached to the email.

---

13. DOCUMENT SECURITY

Before attaching a document:

1. Check that the document exists.
2. Check that the document belongs to the authenticated user.
3. Generate/use the correct PDF.
4. Attach only the allowed document.
5. Never expose internal file paths.

A user must never be able to attach another user's private document by changing a document ID.

---

14. EMAIL HISTORY

Add/maintain email history.

Store:

```
user_id
recipient_email
sender_name
sender_email
subject
document_id
sent_at
status
```

Possible statuses:

```
Sent
Failed
```

Do NOT store:

```
SMTP_PASSWORD
Gmail password
Google App Password
```

Normal users should only see their own email history.

Admins can access email history according to the existing admin authorization system.

---

15. ERROR HANDLING

If Gmail configuration is missing:

Return HTTP 503 and show:

```
Email service is not configured. Please contact the administrator.
```

If Gmail authentication fails:

```
Unable to authenticate with the email service. Please contact the
administrator.
```

If recipient email is invalid:

```
Please enter a valid recipient email address.
```

If sender email is invalid:

```
Please enter a valid sender email address.
```

If the PDF/document does not exist:

```
The selected document could not be found.
```

If SMTP connection fails:

```
Unable to connect to the email service. Please try again later.
```

If sending fails:

```
Unable to send the email. Please try again.
```

Never expose:

- SMTP password
- App Password
- SMTP authentication details

- Python stack traces
- Internal file paths
- Sensitive server information

to the user.

---

16. SECURITY

Implement:

- Authentication
- Authorization
- Backend validation
- Email validation
- Input sanitization
- Header injection protection
- Attachment validation
- File ownership validation
- TLS
- Secure environment variables
- No secrets in frontend
- No secrets in logs
- No secrets in database

Never allow a user to change:

```
SMTP_USERNAME
SMTP_FROM_EMAIL
SMTP_FROM_NAME
```

from the frontend.

---

17. REMOVE OLD RESEND UI

If the application currently shows:

```
Set up Resend API
API Key
From Email
```

REMOVE THAT ENTIRE UI.

Users should NOT see:

```
Resend API Key
Resend From Email
Resend Connector
Configure Resend
```

Instead, email configuration should be an administrator/backend environment configuration.

Normal users and admins should simply see:

```
Send Email
```

They should never have to configure the email provider.

---

18. ENVIRONMENT CONFIGURATION UI

Do not create a frontend page for entering:

```
SMTP_PASSWORD
```

or any other SMTP secret.

The administrator should configure these through the server's environment/secrets:

```
SMTP_HOST
SMTP_PORT
SMTP_USERNAME
SMTP_PASSWORD
SMTP_FROM_EMAIL
SMTP_FROM_NAME
```

The frontend should only use the email functionality.

---

19. EMAIL CONFIGURATION STATUS

Add a safe backend configuration check.

It may report:

```
SMTP Configuration:
Configured
```

or:

```
SMTP Configuration:
Not Configured
```

But NEVER display the actual values.

For diagnostics, it may safely report:

```
SMTP_HOST: configured
SMTP_USERNAME: configured
SMTP_PASSWORD: configured
SMTP_FROM_EMAIL: configured
```

Never display their values.

---

20. USER EXPERIENCE

Normal user:

```
Login
 ↓
Dashboard
 ↓
Send Email
 ↓
Enter Recipient
 ↓
Enter Sender Name
 ↓
Enter Sender Email
 ↓
Enter Subject
 ↓
Enter Message
 ↓
Select PDF
 ↓
Send Email
 ↓
Gmail SMTP
 ↓
Recipient
```

Admin:

```
Login
 ↓
Admin Dashboard
 ↓
Send Email
 ↓
Enter Recipient
 ↓
Enter Sender Details
 ↓
Enter Message
 ↓
Send
 ↓
Gmail SMTP
 ↓
Recipient
```

---

21. EXAMPLE FINAL EMAIL

If the user enters:

```
Recipient:
lawyer@example.com

Sender Name:
Pati Sravya

Sender Email:
sravya@gmail.com

Subject:
Legal Agreement

Message:
Please find the requested agreement attached.
```

The recipient must receive:

```
From:
AI Legal <website-email@gmail.com>

To:
lawyer@example.com
```

```
Subject:
Legal Agreement
```

Email body:

```
AI LEGAL

Sender Details

Name: Pati Sravya
Email: sravya@gmail.com

Message

Please find the requested agreement attached.

Attachment:
Legal_Agreement.pdf
```

---

22. DO NOT BREAK EXISTING APPLICATION

Preserve all existing:

- Registration
- Login
- User authentication
- Admin authentication
- User dashboard
- Admin dashboard
- AI chat
- Document upload
- Legal document analysis
- Legal document generation
- PDF generation
- Database
- Location functionality
- Audit logs
- Existing routes
- Existing APIs
- Existing UI

Only replace the email provider/integration.

---

23. FINAL CLEANUP

After implementing Gmail SMTP:

Search the complete project for:

```
Resend
RESEND_API_KEY
RESEND_FROM_EMAIL
resend
```

Remove all obsolete email-related references.

The final project must have NO dependency on Resend.

Search again and verify there are no active Resend API calls.

Also check:

```
requirements.txt
package files
.env
Replit configuration
frontend
backend
services
templates
static
```

Remove unused Resend dependencies where appropriate.

---

24. FINAL TESTING

Test all of these:

Test 1 — Normal User

Normal user sends email.

Expected:

```
Email sent successfully.
```

Test 2 — Admin

Admin sends email.

Expected:

```
Email sent successfully.
```

Test 3 — PDF

User sends generated legal PDF.

Expected:

```
Email delivered with PDF attachment.
```

Test 4 — Sender Information

Recipient sees:

```
Sender Name
Sender Email
```

inside the email body.

Test 5 — Actual Sender

Recipient sees:

```
From: AI Legal <website-email@gmail.com>
```

The user's email must NOT appear as the actual From address.

Test 6 — Invalid Email

Invalid recipient must be rejected.

Test 7 — Missing Configuration

Missing SMTP credentials must produce a safe 503 configuration error.

Test 8 — Security

Verify:

- SMTP password is never exposed.

- App Password is never exposed.
- Users cannot change the website sender.
- Users cannot access other users' documents.
- Users cannot access other users' email history.
- No Resend API key is required.
- No Resend API calls remain.

FINAL OBJECTIVE

The final email architecture must be:

AI Legal Website → Flask Backend → Gmail SMTP → Website Gmail Account → Any valid recipient

Completely remove Resend and make Gmail SMTP the only email delivery mechanism.